

Memory

2026 Semester 2 COMPSCI 340: Operating Systems
Talía Xu

Lecture 9
2.1.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

How Can Every Process See Its Own Memory?

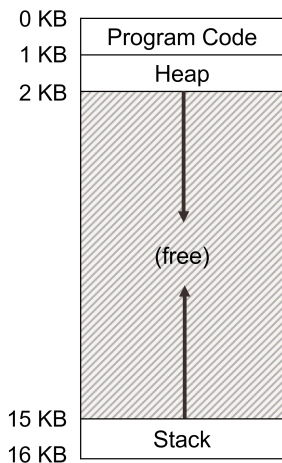
Many processes share one physical memory, but each process should behave as though its memory were private.

This lecture builds that abstraction one step at a time:

physical memory → address spaces → hardware translation → paging →
page tables

By the end, we will translate a virtual address into a physical address and explain why its offset does not change.

An Address Space Is One Process's View of Memory



An address space is the memory layout seen by one running process.

Code: where instructions live

Heap: memory requested while the program runs, such as with `malloc`

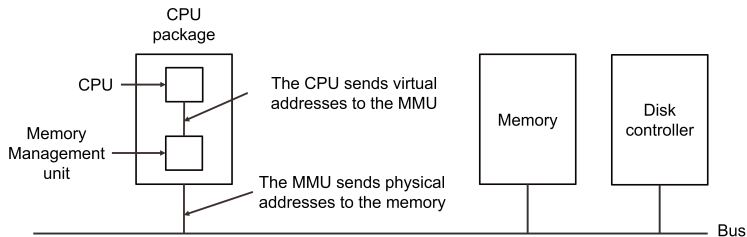
Stack: function calls, local variables, arguments, and return addresses

A Pointer Contains an Address from the Process's View

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    printf("location of code : %p\n", main);
    printf("location of heap : %p\n", malloc(100e3));
    int x = 3;
    printf("location of stack: %p\n", &x);
    return x;
}
```

Are the printed addresses virtual addresses or physical addresses?

The MMU Translates Each Address Before RAM Is Accessed



The program produces a virtual address. The memory management unit (MMU) translates it into the physical address sent to RAM.

Virtual Memory Must Provide Isolation Without Being Slow

- Let many processes share physical memory
- Give each process a private view of memory
- Prevent access to another process's data unless sharing is allowed
- Keep translation fast
- Avoid wasting too much physical memory through fragmentation

A Memory Address Selects One Byte

Each memory address identifies one byte. A larger value occupies several neighbouring byte addresses.

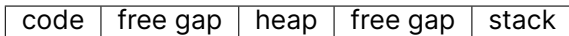
100	101	102	103	104	105	106	107
one 4-byte int				next four bytes			

Segmentation Maps a Few Variable-Size Regions

Segmentation gives code, data, heap, and stack their own variable-size regions.

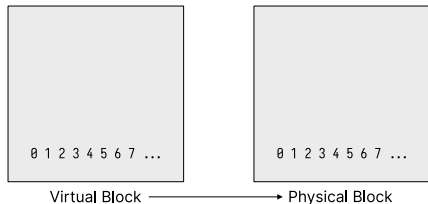
A segment can grow or shrink, but large variable-size regions are costly to move.

As segments are created and resized, small unusable gaps can appear between them. This is **external fragmentation**:



Modern general-purpose operating systems mainly use paging instead.

First Insight: Divide Memory into Fixed-Sized Chunks



Fixed-size blocks can be placed independently, so a process no longer needs one continuous region of physical memory.

Paging

Paging divides memory into equal-size blocks, commonly 4096 bytes (4 KiB).

- A block in virtual memory is a **page**
- A block in physical memory is a **frame**

A page table records which frame currently holds each virtual page.

A Virtual Address Identifies a Page and a Byte

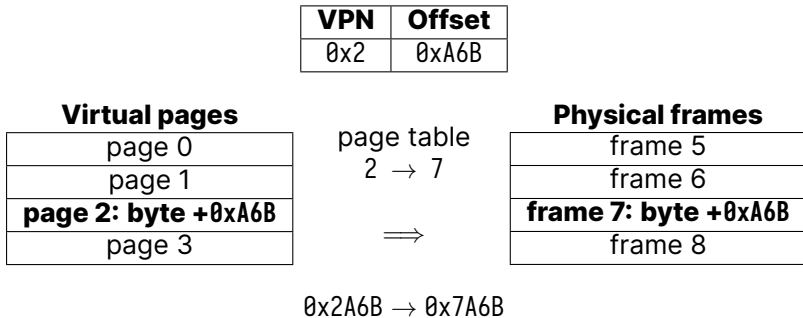
Paging divides an address into two fields:

Virtual page number (VPN)	Offset within the page
selects a page-table entry	selects one byte in that page

The page table replaces the VPN with a physical page number (PPN). The offset does not change because a virtual page and a physical frame have the same size.

One Byte Keeps the Same Offset Inside Its Frame

With 4 KiB pages, each page is 0x1000 bytes. The last three hexadecimal digits therefore identify the byte within the page.



Only the page number changes; 0xA6B still selects the same byte position inside the frame.

Our Examples Use a 39-Bit Virtual Address

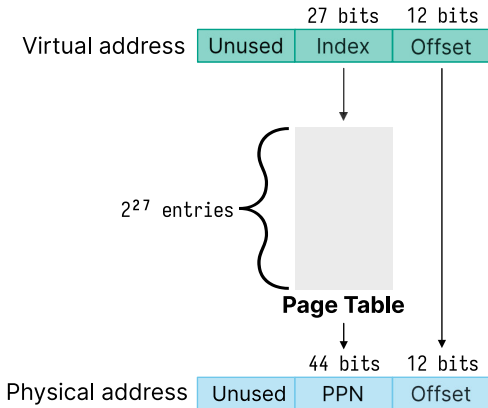
A 64-bit CPU does not necessarily use all 64 bits for virtual addresses.

We will use RISC-V Sv39: a 39-bit virtual address and 4 KiB pages.

A 39-bit address space contains 2^{39} bytes = 512 GiB.

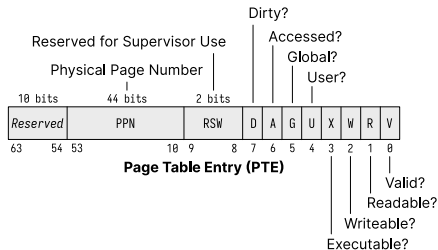
A 4 KiB page needs 12 offset bits, leaving 27 bits for the virtual page number (VPN). The page table uses that VPN to find a physical page number (PPN).

The Page Table Translates Virtual to Physical Addresses



The VPN selects one page-table entry; the offset selects one byte in the chosen physical frame.

A Real RISC-V PTE Packs More Information into 64 Bits



The remaining flags support operating-system bookkeeping and other translation features.

Use the Page Table to Translate Four Addresses

Considering the following page table:

VPN	PPN
0x0	0x1
0x1	0x4
0x2	0x3
0x3	0x7

We would get the following virtual → physical address translations:

0x0AB0 →
0x1FA0 →
0x2884 →
0x32D0 →

Keep the Offset and Replace Only the Page Number

With 4 KiB pages, the last three hexadecimal digits are the 12-bit offset.

Virtual address	VPN lookup	PPN	Physical address
0x0 AB0	0x0 → 0x1	0x1	0x1 AB0
0x1 FA0	0x1 → 0x4	0x4	0x4 FA0
0x2 884	0x2 → 0x3	0x3	0x3 884
0x3 2D0	0x3 → 0x7	0x7	0x7 2D0

The bold offset is copied unchanged into the physical address.

Page Translation Example Problem

Assume you have a 8-bit virtual address, 10-bit physical address and each page is 64 bytes

- How many virtual pages are there?
- How many physical frames are there?
- How many entries are in the page table?
- Given page-table entries 0–3 contain [0x2, 0x5, 0x1, 0x8]
what's the physical address of 0xF1?

Page Translation Example: Worked Answer

A 64-byte page uses 6 offset bits. The 8-bit virtual address therefore has 2 VPN bits:

$$\begin{array}{ccc} 0xF1 & = & 11 \mid 110001 \\ \text{VPN} & & \text{offset} \end{array}$$

- Virtual pages: $2^8/2^6 = 4$
- Physical frames: $2^{10}/2^6 = 16$
- Page-table entries: 4
- VPN 11 is entry 3, which contains PPN 0x8

$$\text{PPN } 001000 \mid \text{offset } 110001 = 0x231$$

A Single-Level Page Table Can Be Mostly Empty but Still Huge

The 39-bit virtual address uses a 12-bit page offset, leaving a 27-bit VPN.

$$2^{27} \text{ entries} \times 8 \text{ bytes per entry} = 2^{30} \text{ bytes} = 1 \text{ GiB}$$

A small process may use only a few of those virtual pages, but a flat table still reserves an entry for every possible VPN.

The next step is to divide the page table itself into levels, so unused regions do not need lower-level tables.

We Use Pages for Memory Translation

Paging divides virtual and physical memory into equal-size blocks.

A virtual address contains a VPN and an offset. The page table replaces the VPN with a PPN and keeps the offset unchanged.

A flat page table is simple, but it reserves an entry for every possible virtual page. Next, we make page tables smaller and translation faster.